

Object Oriented Programming (CS34)

Academic year 2024 - 2025

Faculty : Jamuna S Murthy

Method Overloading

Easy:

1. Calculator Operations

Create a `Calculator` class with overloaded methods for addition, subtraction, multiplication, and division. Each operation should be overloaded to handle both two and three parameters. For example, `add(int a, int b)` and `add(int a, int b, int c)`.

2. Area Calculator

Create an `AreaCalculator` class with overloaded methods named `calculateArea` to find the area of different shapes. Include methods to calculate the area of:

- A square (takes one `int` for the side length)
- A rectangle (takes two `int` values for length and width)
- A circle (takes one `double` for radius)

3. Volume Calculator

Design a `VolumeCalculator` class with overloaded methods called `calculateVolume` to find the volume of:

- A cube (takes one `int` for side length)
- A rectangular prism (takes three `int` values for length, width, and height)
- A cylinder (takes two `double` values for radius and height)

4. Display Information

Write a `Display` class with overloaded methods named `showInfo`. Overload `showInfo` to display different types of information based on parameters:

- One `String` parameter for displaying a name
- One `String` and one `int` for displaying a name and age
- One `String`, one `int`, and one `String` for displaying name, age, and occupation

5. Converter Utility

Create a `Converter` class with overloaded methods to convert:

- Meters to kilometers (takes one `double` for meters)
- Kilometers to miles (takes one `double` for kilometers)
- Celsius to Fahrenheit (takes one `double` for Celsius temperature)

Intermediate:

1. Statistics Calculator

Create a `Statistics` class with overloaded methods called `calculateAverage`. Each method should calculate the average of:

- Two `int` values
- An array of `int` values
- An array of `double` values
- A variable number of `int` arguments using varargs (e.g., `calculateAverage(int... values)`)

2. Matrix Operations

Create a `MatrixOperations` class with overloaded methods for adding matrices. Each overloaded `addMatrices` method should handle:

- Two 2x2 matrices represented as 2D arrays of `int`
- Two 3x3 matrices represented as 2D arrays of `int`
- Two matrices of any dimensions (where both matrices must have the same dimensions) represented as 2D arrays of `int`

3. Person Information Display

Write a `PersonInfo` class with overloaded methods named `displayDetails` that displays information based on the input:

- One `String` representing the name
- `String` name and `int` age
- `String` name, `int` age, and a `String` address
- `String` name, `int` age, `String` address, and an array of `String` representing hobbies

4. Bank Account Transactions

Create a `BankAccount` class with overloaded methods called `withdraw` to simulate different withdrawal methods:

- Withdraw using an `int` amount
- Withdraw using an `int` amount and a `String` password
- Withdraw using an `int` amount, `String` password, and a `String` OTP (One-Time Password)
- Withdraw using a `double` amount (for cases where the account balance includes cents or fractions)

5. Unit Converter

Design a `UnitConverter` class with overloaded methods named `convert` to handle different types of conversions:

- Convert `double` kilometers to miles
- Convert `double` pounds to kilograms
- Convert `int` minutes to hours and minutes (returns a formatted `String`)
- Convert an array of `double` temperatures in Celsius to Fahrenheit and return them as an array of `double`

Hard:

Library Management System

Problem Statement:

Design a **Library** management system with method overloading to handle different types of operations for managing books, patrons, and transactions. This system should include overloaded methods to:

1. Add books and patrons.
2. Search for books by different criteria.
3. Issue books to patrons based on various conditions.
4. Return books and calculate fines based on the return date.

Each class should include multiple overloaded methods to handle these operations based on different criteria.

Requirements

1. Book Class:

- Attributes: `bookId`, `title`, `author`, `yearPublished`, `isAvailable`.
- Methods:
 - Overloaded `searchBook()`:
 - By `String title`
 - By `String author` and `int yearPublished`
 - By `int bookId`
 - Overloaded `issueBook()`:
 - By `bookId` and `patronId`
 - By `bookId`, `patronId`, and `int days` (for limited-time issuance)

2. Patron Class:

- Attributes: `patronId`, `name`, `membershipType` (e.g., Regular, Premium), `booksIssued` (list of issued books).
- Methods:
 - Overloaded `addPatron()`:
 - By `String name` and `String membershipType`

- By `String name`, `String membershipType`, and `int age`
- `viewIssuedBooks()` (displays all books currently issued to the patron)

3. Library Class:

- Attributes: List of `Book` objects, list of `Patron` objects.
- Methods:
 - Overloaded `addBook()`:
 - By `String title`, `String author`, `int yearPublished`
 - By `Book book` (accepts a pre-created Book object)
 - Overloaded `returnBook()`:
 - By `bookId` and `patronId`
 - By `bookId`, `patronId`, and `boolean isLate` (to calculate fines for late returns)